Caso 1: gestión de flota y mantenimiento

Trabajo Práctico 2, segunda etapa: funciones logísticas y aplicación web

---

Alumnas: Yanila Jazmin Benítez; Ruth Maillén Martinez
Informática Aplicada
Tecnicatura Universitaria en Logística, UTN
Octubre de 2026

1. Funciones logísticas que el modelo de la etapa 1 no cubre

El modelo de la etapa 1 registra qué pasó (salidas, chequeos, mantenimientos o reservas, limpiezas, inspecciones). No ayuda a anticiparse, a controlar plazos ni a medir. Se identificaron cuatro funciones logísticas faltantes; las tres primeras agregan datos al modelo y la cuarta se resuelve con los datos existentes.

Función	Problema	Usuario	Dato que necesita	Decisión de modelado
F1. Plan de mantenimiento preventivo por kilometraje	El modelo de la etapa 1 registra mantenimientos preventivos, pero nada indica cuándo corresponde hacerlos. Los services se hacen por memoria o tarde.	Jefe de taller	Kilometraje actual de cada vehículo, intervalo de cada plan (por ejemplo cada 10.000 km) y kilometraje del último preventivo.	Se agrega el kilometraje acumulado al vehículo, que se actualiza solo al cerrar cada salida, y una entidad PLAN_PREVENTIVO. Una vista lista los planes vencidos y una función abre el mantenimiento preventivo desde el plan. No se genera el mantenimiento de forma automática porque requiere asignar un mecánico.
F2. Control de stock del pañol con movimientos y punto de reposición	La etapa 1 solo descuenta stock al usar un repuesto. No hay registro de ingresos, ni de quién retiró qué, ni aviso cuando un repuesto se está agotando.	Pañolero	Cada movimiento (ingreso, consumo, ajuste) con fecha, cantidad y mantenimiento de origen, y un stock mínimo por repuesto.	Se agrega la entidad MOVIMIENTO_STOCK y el atributo stock_minimo. El stock cambia únicamente a través de movimientos, lo que da trazabilidad completa. Una vista muestra los repuestos en o bajo el mínimo.
F3. Alertas de vencimiento con seguimiento	La vista de la etapa 1 muestra vencimientos próximos, pero no queda registro de quién los vio ni si se resolvieron. Un vencimiento puede avisarse una y otra vez o pasar inadvertido.	Supervisor de flota	Alerta por documento o licencia, con fecha de generación, estado (pendiente o atendida), quién la atendió y cuándo.	Se agrega la entidad ALERTA_VENCIMIENTO, que se genera una sola vez por vencimiento y se marca como atendida. Una alerta apunta a un documento o a una licencia de chofer, nunca a ambos.
F4. Indicadores de flota	Quien supervisa no puede ver de un vistazo el estado de la flota: cuántos vehículos están operativos, cuántas salidas se bloquearon o cuántos correctivos tuvo cada unidad.	Supervisor de flota	Datos que ya existen: estado, kilometraje, salidas y mantenimientos por vehículo.	No requiere entidades nuevas. Se resuelve con una vista de indicadores por vehículo y un tablero en la aplicación.

2. Documento descriptivo

Este documento se usó como prompt para generar la aplicación (sección 6). Todos los datos son inventados.

1. Contexto de la empresa

Una empresa minera de alta montaña opera una flota de camionetas, camiones y buses que se desplazan entre San Juan, Batidero, Rodeo y La Guanaca. Cada vehículo pertenece a un área de la empresa y sale a ruta con un chofer. Hoy los vencimientos de licencias y documentación no se controlan de forma sistemática, hay salidas sin chequeo previo registrado y el mantenimiento no tiene trazabilidad ni control de repuestos. El sistema busca que ningún vehículo salga sin chequeo, documentación y licencia vigentes, que el mantenimiento preventivo se haga a tiempo y que el pañol tenga el stock bajo control.

2. Actores y roles

- **Supervisor de flota.** Solicita salidas, revisa el estado de la flota, genera y atiende alertas de vencimiento y consulta los indicadores.
- **Chofer.** Registra el chequeo previo de su vehículo, inicia la salida si el chequeo fue aprobado y la cierra al regresar informando los kilómetros.
- **Jefe de taller.** Abre y cierra mantenimientos correctivos y preventivos, asigna mecánicos y administra los planes preventivos por kilometraje.
- **Pañolero.** Registra ingresos de repuestos y las entregas a los mantenimientos, y controla el stock y los puntos de reposición.
- **Administrador.** Mantiene los datos maestros: vehículos, áreas, choferes y documentación.

3. Datos del negocio según el nuevo MER

Las entidades de la etapa 1 se mantienen. Se incorporan las siguientes (ver sección 3 y el diagrama):

- **VEHICULO** (modificada). Se agrega km_actual, necesario para saber cuándo corresponde un preventivo. Se actualiza al cerrar cada salida.
- **REPUESTO** (modificada). Se agrega stock_minimo, que define cuándo hay que reponer.
- **PLAN_PREVENTIVO** (nueva). Guarda cada plan de mantenimiento por kilometraje de un vehículo (descripción, intervalo y último kilometraje atendido).
- **MOVIMIENTO_STOCK** (nueva). Registra cada ingreso, consumo o ajuste de repuestos; es la única vía para cambiar el stock.
- **ALERTA_VENCIMIENTO** (nueva). Guarda las alertas de vencimiento y quién las atendió.

4. Funciones logísticas

- **F1. Plan de mantenimiento preventivo por kilometraje** Problema: El modelo de la etapa 1 registra mantenimientos preventivos, pero nada indica cuándo corresponde hacerlos. Los services se hacen por memoria o tarde. Usuario: Jefe de taller. Datos: Kilometraje actual de cada vehículo, intervalo de cada plan (por ejemplo cada 10.000 km) y kilometraje del último preventivo. Decisión: Se agrega el kilometraje acumulado al vehículo, que se actualiza solo al cerrar cada salida, y una entidad PLAN_PREVENTIVO. Una vista lista los planes vencidos y una función abre el mantenimiento preventivo desde el plan. No se genera el mantenimiento de forma automática porque requiere asignar un mecánico.
- **F2. Control de stock del pañol con movimientos y punto de reposición** Problema: La etapa 1 solo descuenta stock al usar un repuesto. No hay registro de ingresos, ni de quién retiró qué, ni aviso cuando un repuesto se está agotando. Usuario: Pañolero. Datos: Cada movimiento (ingreso, consumo, ajuste) con fecha, cantidad y mantenimiento de origen, y un stock mínimo por repuesto. Decisión: Se agrega la entidad MOVIMIENTO_STOCK y el atributo stock_minimo. El stock cambia únicamente a través de movimientos, lo que da trazabilidad completa. Una vista muestra los repuestos en o bajo el mínimo.
- **F3. Alertas de vencimiento con seguimiento** Problema: La vista de la etapa 1 muestra vencimientos próximos, pero no queda registro de quién los vio ni si se resolvieron. Un vencimiento puede avisarse una y otra vez o pasar inadvertido. Usuario: Supervisor de flota. Datos: Alerta por

documento o licencia, con fecha de generación, estado (pendiente o atendida), quién la atendió y cuándo. Decisión: Se agrega la entidad ALERTA_VENCIMIENTO, que se genera una sola vez por vencimiento y se marca como atendida. Una alerta apunta a un documento o a una licencia de chofer, nunca a ambos.

- **F4. Indicadores de flota** Problema: Quien supervisa no puede ver de un vistazo el estado de la flota: cuántos vehículos están operativos, cuántas salidas se bloquearon o cuántos correctivos tuvo cada unidad. Usuario: Supervisor de flota. Datos: Datos que ya existen: estado, kilometraje, salidas y mantenimientos por vehículo. Decisión: No requiere entidades nuevas. Se resuelve con una vista de indicadores por vehículo y un tablero en la aplicación.

5. Reglas del negocio

1. Toda salida nace en estado solicitada y solo puede pasar a en curso, bloqueada o finalizada en el orden definido.
2. Una salida solo puede iniciarse si tiene un chequeo aprobado, el vehículo está operativo, tiene seguro, VTV y habilitación de altura vigentes y la licencia del chofer no está vencida.
3. El resultado del chequeo se calcula a partir de sus ítems: si falla uno, el chequeo queda rechazado, la salida se bloquea y el vehículo pasa a fuera de servicio.
4. Un vehículo o un chofer no pueden estar en dos salidas al mismo tiempo.
5. Al abrir un mantenimiento el vehículo pasa a en mantenimiento; al cerrarlo vuelve a operativo si no tiene otros abiertos.
6. Al cerrar una salida, los kilómetros recorridos se suman al kilometraje actual del vehículo.
7. Un plan preventivo vence cuando el kilometraje actual menos el kilometraje del último preventivo alcanza el intervalo del plan.
8. El stock de un repuesto solo cambia mediante movimientos; nunca puede quedar negativo.
9. Un repuesto está en reposición cuando su stock es menor o igual a su stock mínimo.
10. Una alerta de vencimiento se genera una sola vez por documento o licencia y vencimiento, y puede atenderse una sola vez.

6. Pantallas y reportes por rol

Rol	Pantalla	Contenido
Supervisor de flota	Tablero	Vehículos por estado, salidas en curso, alertas pendientes, preventivos pendientes, repuestos bajo el mínimo e indicadores por vehículo.
Supervisor de flota	Salidas	Solicitud de salida (vehículo, chofer, origen, destino) y listado de salidas con su estado.
Supervisor de flota	Vehículos	Listado de la flota con área, estado y kilometraje.
Supervisor de flota	Alertas	Generación de alertas a 30 días y atención de cada una.
Chofer	Mis salidas	Salidas del chofer, formulario de chequeo previo, inicio de salida y cierre con kilómetros.
Jefe de taller	Mantenimientos	Apertura de mantenimientos, listado y cierre.
Jefe de taller	Preventivos	Planes vencidos por kilometraje con apertura de preventivo, y alta de planes.
Pañolero	Repuestos	Stock con aviso de reposición, ingreso de repuestos, entrega a mantenimientos y últimos movimientos.
Administrador	Maestros	Alta de vehículos, carga y renovación de documentación, listado de choferes con su licencia.

7. Datos de ejemplo (ficticios)

- Vehículos: AA111AA (camioneta, área Operaciones de mina), AA333CC (camión con VTV vencida), AA444DD (bus con seguro por vencer), AA555EE (camioneta con chequeo rechazado).
- Choferes: CH001 y CH004 con licencia vigente; CH002 con licencia vencida; CH003 con licencia por vencer en 15 días. Los nombres son inventados (Chofer Ficticio Uno, etc.).

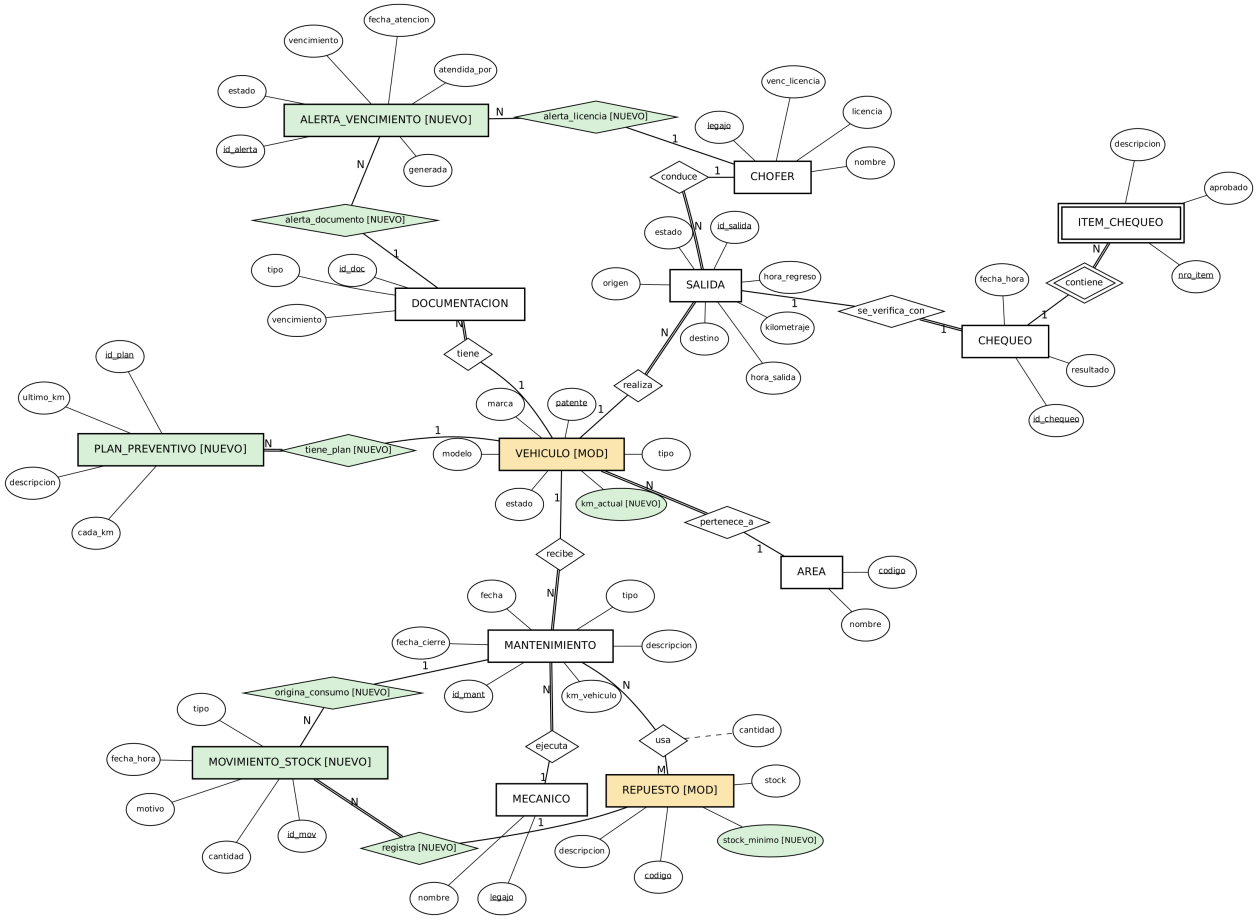
- Mecánicos: ME001 y ME002. Repuestos: REP-FILT-01 (filtro de aceite), REP-PAST-02 (pastillas de freno), REP-ACEI-03 (aceite de motor).
- Planes preventivos: service de 10.000 km para AA111AA y AA333CC, cambio de neumáticos cada 40.000 km para AA444DD. AA333CC tiene 12.500 km y tiene el service vencido.
- Una salida finalizada de AA222BB de 180 km, una salida solicitada que espera chequeo y una salida bloqueada con el vehículo en mantenimiento correctivo.

8. Requisitos técnicos

- Aplicación web de una sola página con React, TypeScript y Vite, en español, con selector de rol y una pantalla por función.
- Base de datos en Supabase (PostgreSQL). Las reglas del negocio viven en la base (restricciones y triggers). La aplicación consume tablas, vistas y funciones (RPC) mediante supabase-js.
- Seguridad por filas (RLS) habilitada en todas las tablas. En esta versión las políticas son abiertas para la demostración y el rol se elige en pantalla; para producción corresponde usar Supabase Auth y políticas por rol.
- Despliegue en Vercel como sitio estático. Las variables VITE_SUPABASE_URL y VITE_SUPABASE_ANON_KEY se cargan en Vercel y en un archivo .env local que no se sube al repositorio.
- Repositorio en GitHub con el código, la carpeta docs, el README, un .env.example sin valores y un .gitignore que excluye .env.
- Todos los datos son inventados. No se usan DNI, tarjetas ni nombres de personas reales.

3. Nuevo MER

Notación de Chen. En verde figuran las entidades y relaciones nuevas; en ámbar, las entidades modificadas, con el atributo agregado marcado. Cada elemento lleva además la etiqueta [NUEVO] o [MOD] para que se distinga también en blanco y negro.



Nuevo MER, Caso 1, flota y mantenimiento

3.1 Cambios respecto del MER de la etapa 1 y su justificación

Elemento	Tipo de cambio	Por qué
VEHICULO	Modificada	Se agrega km_actual, necesario para saber cuándo corresponde un preventivo. Se actualiza al cerrar cada salida.
REPUESTO	Modificada	Se agrega stock_minimo, que define cuándo hay que reponer.
PLAN_PREVENTIVO	Nueva	Guarda cada plan de mantenimiento por kilometraje de un vehículo (descripción, intervalo y último kilometraje atendido).
MOVIMIENTO_STOCK	Nueva	Registra cada ingreso, consumo o ajuste de repuestos; es la única vía para cambiar el stock.
ALERTA_VENCIMIENTO	Nueva	Guarda las alertas de vencimiento y quién las atendió.
tiene_plan (VEHICULO 1:N PLAN_PREVENTIVO)	Nueva	Un vehículo puede tener varios planes; todo plan pertenece a un vehículo.
registra (REPUESTO 1:N MOVIMIENTO_STOCK)	Nueva	Todo movimiento afecta a un repuesto.
origina_consumo (MANTENIMIENTO 1:N MOVIMIENTO_STOCK)	Nueva	Un consumo puede originarse en un mantenimiento; los ingresos no tienen mantenimiento (participación parcial).
alerta_documento (DOCUMENTACION 1:N ALERTA_VENCIMIENTO)	Nueva	Una alerta puede referirse a un documento.
alerta_licencia (CHOFER 1:N ALERTA_VENCIMIENTO)	Nueva	Una alerta puede referirse a la licencia de un chofer. Una alerta apunta a uno solo de los dos.

4. Modelo relacional y SQL

4.1 Pedido a la IA

Herramienta utilizada: Claude. Se le entregaron el documento descriptivo de la sección 2, el MER nuevo y el script de la etapa 1 ya corregido.

```
A partir del documento descriptivo, del MER nuevo (con lo nuevo y lo modificado marcado) y del script SQL
de la etapa 1 que ya existe en la base, generá el modelo relacional de los cambios y un script SQL para
PostgreSQL/Supabase. Condiciones: no borres ni recrees lo que ya existe; hacé los cambios con ALTER y CREATE;
respetá las claves y los nombres existentes; hacé cumplir las reglas del negocio dentro de la base (CHECK,
claves foráneas, triggers); activá RLS en cada tabla nueva; creá las funciones y vistas que necesite la
aplicación; migrá los datos existentes si cambia la forma de guardar algo; usá solo datos ficticios.
```

4.2 Respuesta de la IA

La IA devolvió una migración aditiva con tablas, columnas, triggers, vistas y funciones nuevas. Se transcribe completa.

```
-- ======================================================================
-- Caso 1: Gestión de flota y mantenimiento. Etapa 2.
-- Migración aditiva sobre la base de la etapa 1 (no se recrea la base).
-- Agrega: plan de mantenimiento preventivo por kilometraje, control de stock
-- con movimientos, alertas de vencimiento con seguimiento e indicadores.
-- Todos los datos de prueba son ficticios.
-- ======================================================================

-- ---------- F1. Plan de mantenimiento preventivo por kilometraje ----------
alter table vehiculo add column km_actual numeric(9, 1) not null default 0 check (km_actual >= 0);

create table plan_preventivo (
  id_plan     uuid primary key default gen_random_uuid(),
  patente     text not null references vehiculo (patente),
  descripcion text not null,
  cada_km     numeric(8, 1) not null check (cada_km > 0),
  ultimo_km   numeric(9, 1) not null default 0 check (ultimo_km >= 0),
  unique (patente, descripcion)
);
create index on plan_preventivo (patente);

-- Al cerrar una salida se suma el kilometraje recorrido al vehículo.
create function fn_sumar_km_vehiculo() returns trigger
language plpgsql set search_path = public as $$
begin
  if new.estado = 'finalizada' and old.estado = 'en_curso' then
    update vehiculo set km_actual = km_actual + coalesce(new.kilometraje, 0) where patente = new.patente;
  end if;
  return null;
end $$;

create trigger trg_salida_suma_km
  after update of estado on salida
  for each row execute function fn_sumar_km_vehiculo();

create view v_preventivos_pendientes with (security_invoker = true) as
  select p.id_plan, p.patente, p.descripcion, p.cada_km, p.ultimo_km, v.km_actual,
         (v.km_actual - p.ultimo_km) as km_desde_ultimo,
         (v.km_actual - p.ultimo_km - p.cada_km) as km_excedidos
    from plan_preventivo p join vehiculo v using (patente)
   where v.km_actual - p.ultimo_km >= p.cada_km
   order by km_excedidos desc;

create function abrir_mantenimiento_preventivo(p_plan uuid, p_legajo text) returns uuid
language plpgsql set search_path = public as $$
declare
  v_plan plan_preventivo%rowtype;
  v_km   numeric;
  v_id   uuid;
begin
  select * into v_plan from plan_preventivo where id_plan = p_plan;
  if not found then raise exception 'El plan preventivo no existe' using errcode = 'no_data_found'; end if;
  select km_actual into v_km from vehiculo where patente = v_plan.patente;
  insert into mantenimiento (patente, legajo_mecanico, fecha, tipo, descripcion, km_vehiculo)
  values (v_plan.patente, p_legajo, current_date, 'preventivo', v_plan.descripcion, v_km)
  returning id_mant into v_id;
  update plan_preventivo set ultimo_km = v_km where id_plan = p_plan;
  return v_id;
end $$;

-- ---------- F2. Control de stock con movimientos y punto de reposición ----------
alter table repuesto add column stock_minimo integer not null default 0 check (stock_minimo >= 0);
```

```
create table movimiento_stock (
  id_mov     uuid primary key default gen_random_uuid(),
  codigo     text not null references repuesto (codigo),
  id_mant    uuid references mantenimiento (id_mant) on delete set null,
  fecha_hora timestamptz not null default now(),
  tipo       text not null check (tipo in ('ingreso', 'consumo', 'ajuste')),
  cantidad   integer not null check (cantidad <> 0),
  motivo     text,
  check (tipo <> 'ingreso' or cantidad > 0),
  check (tipo <> 'consumo' or cantidad < 0)
);
create index on movimiento_stock (codigo);
create index on movimiento_stock (id_mant);

-- Los movimientos son la única vía para cambiar el stock.
create function fn_aplicar_movimiento() returns trigger
language plpgsql set search_path = public as $$
begin
  update repuesto set stock = stock + new.cantidad where codigo = new.codigo;
  return null;
end $$;

create trigger trg_movimiento_aplica
  after insert on movimiento_stock
  for each row execute function fn_aplicar_movimiento();

-- El consumo en un mantenimiento ahora genera su movimiento (reemplaza el descuento directo de la etapa 1).
create or replace function fn_descontar_stock() returns trigger
language plpgsql set search_path = public as $$
begin
  if tg_op = 'INSERT' then
    insert into movimiento_stock (codigo, id_mant, tipo, cantidad, motivo)
    values (new.codigo, new.id_mant, 'consumo', -new.cantidad, 'Uso en mantenimiento');
  elsif tg_op = 'UPDATE' then
    if new.cantidad <> old.cantidad then
      insert into movimiento_stock (codigo, id_mant, tipo, cantidad, motivo)
      values (new.codigo, new.id_mant, 'ajuste', old.cantidad - new.cantidad, 'Corrección de cantidad usada');
    end if;
  elsif tg_op = 'DELETE' then
    insert into movimiento_stock (codigo, id_mant, tipo, cantidad, motivo)
    values (old.codigo, null, 'ajuste', old.cantidad, 'Anulación de uso en mantenimiento');
  end if;
  return null;
end $$;

create function ingresar_repuesto(p_codigo text, p_cantidad integer, p_motivo text default 'Ingreso al pañol')
    returns void
language plpgsql set search_path = public as $$
begin
  insert into movimiento_stock (codigo, tipo, cantidad, motivo) values (p_codigo, 'ingreso', p_cantidad, p_motivo);
end $$;

create view v_reposicion with (security_invoker = true) as
  select codigo, descripcion, stock, stock_minimo, (stock_minimo - stock) as faltante
    from repuesto
   where stock <= stock_minimo
   order by faltante desc;

-- ---------- F3. Alertas de vencimiento con seguimiento ----------
create table alerta_vencimiento (
  id_alerta      uuid primary key default gen_random_uuid(),
  id_doc         uuid references documentacion (id_doc) on delete cascade,
  legajo_chofer  text references chofer (legajo) on delete cascade,
  vencimiento    date not null,
  generada       date not null default current_date,
  estado         text not null default 'pendiente' check (estado in ('pendiente', 'atendida')),
  atendida_por   text,
  fecha_atencion date,
  check (num_nonnulls(id_doc, legajo_chofer) = 1),
  check (estado = 'pendiente' or (atendida_por is not null and fecha_atencion is not null))
);
create unique index alerta_doc_unica on alerta_vencimiento (id_doc, vencimiento) where id_doc is not null;
create unique index alerta_lic_unica on alerta_vencimiento (legajo_chofer, vencimiento) where legajo_chofer is not
    null;
create index on alerta_vencimiento (estado);

create function generar_alertas(p_dias integer default 30) returns integer
language plpgsql set search_path = public as $$
declare v_a integer; v_b integer;
begin
  insert into alerta_vencimiento (id_doc, vencimiento)
  select id_doc, vencimiento from documentacion where vencimiento <= current_date + p_dias
  on conflict do nothing;
  get diagnostics v_a = row_count;
  insert into alerta_vencimiento (legajo_chofer, vencimiento)
```

```
  select legajo, venc_licencia from chofer where venc_licencia <= current_date + p_dias
  on conflict do nothing;
  get diagnostics v_b = row_count;
  return v_a + v_b;
end $$;

create function atender_alerta(p_alerta uuid, p_quien text) returns void
language plpgsql set search_path = public as $$
begin
  update alerta_vencimiento set estado = 'atendida', atendida_por = p_quien, fecha_atencion = current_date
   where id_alerta = p_alerta and estado = 'pendiente';
  if not found then raise exception 'La alerta no existe o ya fue atendida' using errcode = 'no_data_found'; end if;
end $$;

-- ---------- F4. Indicadores de flota ----------
create view v_indicadores_vehiculo with (security_invoker = true) as
  select v.patente, v.tipo, v.estado, v.km_actual,
         (select count(*) from salida s where s.patente = v.patente and s.estado = 'finalizada') as
             salidas_finalizadas,
         (select count(*) from salida s where s.patente = v.patente and s.estado = 'bloqueada') as
             salidas_bloqueadas,
         (select count(*) from mantenimiento m where m.patente = v.patente and m.tipo = 'preventivo') as
             preventivos,
         (select count(*) from mantenimiento m where m.patente = v.patente and m.tipo = 'correctivo') as correctivos
    from vehiculo v;

-- ---------- Funciones de la aplicación ----------
create function solicitar_salida(p_patente text, p_legajo text, p_origen text, p_destino text) returns uuid
language plpgsql set search_path = public as $$
declare v_id uuid;
begin
  insert into salida (patente, legajo_chofer, origen, destino) values (p_patente, p_legajo, p_origen, p_destino)
  returning id_salida into v_id;
  return v_id;
end $$;

create function registrar_chequeo(p_salida uuid, p_items jsonb) returns text
language plpgsql set search_path = public as $$
declare v_chq uuid; v_n integer := 0; v_item jsonb; v_res text;
begin
  insert into chequeo (id_salida) values (p_salida) returning id_chequeo into v_chq;
  for v_item in select * from jsonb_array_elements(p_items) loop
    v_n := v_n + 1;
    insert into item_chequeo values (v_chq, v_n, v_item ->> 'descripcion', (v_item ->> 'aprobado')::boolean);
  end loop;
  select resultado into v_res from chequeo where id_chequeo = v_chq;
  return v_res;
end $$;

create function iniciar_salida(p_salida uuid) returns void
language plpgsql set search_path = public as $$
begin
  update salida set estado = 'en_curso' where id_salida = p_salida;
end $$;

create function cerrar_salida(p_salida uuid, p_km numeric) returns void
language plpgsql set search_path = public as $$
begin
  update salida set estado = 'finalizada', hora_regreso = clock_timestamp(), kilometraje = p_km where id_salida =
      p_salida;
end $$;

create function abrir_mantenimiento(p_patente text, p_legajo text, p_tipo text, p_descripcion text) returns uuid
language plpgsql set search_path = public as $$
declare v_id uuid;
begin
  insert into mantenimiento (patente, legajo_mecanico, fecha, tipo, descripcion, km_vehiculo)
  values (p_patente, p_legajo, current_date, p_tipo, p_descripcion, (select km_actual from vehiculo where patente =
      p_patente))
  returning id_mant into v_id;
  return v_id;
end $$;

create function cerrar_mantenimiento(p_mant uuid) returns void
language plpgsql set search_path = public as $$
begin
  update mantenimiento set fecha_cierre = current_date where id_mant = p_mant and fecha_cierre is null;
  if not found then raise exception 'El mantenimiento no existe o ya está cerrado' using errcode = 'no_data_found';
      end if;
end $$;

create function usar_repuesto(p_mant uuid, p_codigo text, p_cantidad integer) returns void
language plpgsql set search_path = public as $$
begin
  insert into mantenimiento_repuesto values (p_mant, p_codigo, p_cantidad)
```

```
  on conflict (id_mant, codigo) do update set cantidad = mantenimiento_repuesto.cantidad + excluded.cantidad;
end $$;

-- ---------- Seguridad (RLS) para las tablas nuevas ----------
do $$
declare t text;
begin
  for t in select tablename from pg_tables where schemaname = 'public' and not rowsecurity loop
    execute format('alter table public.%I enable row level security', t);
    execute format('create policy acceso_demo on public.%I for all to anon, authenticated using (true) with check
        (true)', t);
  end loop;
end $$;
```

4.3 Revisión crítica contra el MER nuevo

Se comprobó cada elemento marcado como nuevo o modificado en la sección 3 contra el esquema creado, y cada regla del documento descriptivo contra una prueba. Resultado: todas las entidades y atributos nuevos existen; las reglas se verificaron con la batería de la sección 4.6. La revisión encontró lo siguiente:

Hallazgo	Qué pasaba	Corrección
Usar dos veces el mismo repuesto en un mantenimiento fallaba	La prueba de repuesto mayor al stock se rechazó por una clave duplicada y no por el stock. La clave (id_mant, codigo) impide registrar dos entregas del mismo repuesto al mismo mantenimiento.	La función usar_repuesto suma a la cantidad ya registrada (INSERT con ON CONFLICT DO UPDATE) y el trigger genera un ajuste por la diferencia.

Diferencias respecto de la propuesta de la IA. La IA reemplazó el descuento directo de stock de la etapa 1 por movimientos y mantuvo un solo punto de cambio de stock. También propuso funciones (RPC) para cada paso del flujo, de modo que la aplicación no tenga que repetir reglas. Se aceptó el uso de vistas con security_invoker para que respeten las políticas de seguridad del usuario que consulta.

4.4 Decisión: recrear o modificar la base

Se decidió modificar la base existente con una migración aditiva y no recrearla. Motivos: (1) la etapa 1 ya tiene datos y evidencia de pruebas que se perderían; (2) los cambios son agregados (dos columnas con valor por defecto, tablas, vistas y funciones nuevas) y un solo reemplazo de función (el descuento de stock); (3) la batería completa de pruebas de la etapa 1 se volvió a ejecutar sobre la base migrada y pasó sin cambios, lo que demuestra que no se rompió nada. Recrear la base solo habría sido necesario si hubiera cambiado el identificador de alguna entidad existente, y no fue el caso.

4.5 Ejecución en Supabase

La migración se ejecutó sobre la base de la etapa 1 ya cargada con datos. Por el límite de dos proyectos del plan gratuito de la organización, la ejecución se hizo en un PostgreSQL 18 equivalente (PGlite) con el mismo script que va en `supabase/migrations`. La base resultante tiene 14 tablas. Para repetir la prueba en un proyecto de Supabase real se ejecutan, en el editor SQL y en este orden, `0001_etapa1.sql`, `0002_etapa2.sql` y `seed.sql`. Datos ficticios cargados para la etapa 2:

```
-- Datos ficticios adicionales, Caso 1, etapa 2.
update repuesto set stock_minimo = 5 where codigo = 'REP-FILT-01';
update repuesto set stock_minimo = 4 where codigo = 'REP-PAST-02';
insert into repuesto (codigo, descripcion, stock, stock_minimo) values ('REP-ACEI-03', 'Aceite de motor 15W40
    (litro)', 40, 20);
insert into plan_preventivo (patente, descripcion, cada_km, ultimo_km) values
  ('AA111AA', 'Service de 10.000 km', 10000, 0),
  ('AA333CC', 'Service de 10.000 km', 10000, 0),
  ('AA444DD', 'Cambio de neumáticos', 40000, 0);
update vehiculo set km_actual = 12500 where patente = 'AA333CC';
update vehiculo set km_actual = 9400 where patente = 'AA111AA';
```

4.6 Pruebas

Regresión: pruebas de la etapa 1 sobre la base migrada

La batería completa de la etapa 1 (pruebas T01 a T16 y de revisión RV01 a RV03) se ejecutó sobre la base ya migrada. Todas pasaron sin modificar ninguna. Primera ejecución sin adaptar: 20 de 20 pruebas pasaron. Ejecución final: 20 de 20.

Prueba	Descripción	Esperado	Obtenido	Mensaje del motor
T01	Alta de vehículo con área	se acepta	aceptado	
T02	Vehículo sin área	se rechaza	rechazado	null value in column "codigo_area" of relation "vehiculo" violates not-null constraint
T03	Tipo de vehículo inexistente	se rechaza	rechazado	new row for relation "vehiculo" violates check constraint "vehiculo_tipo_check"
T04	Salida con chequeo aprobado pasa a en_curso	se acepta	aceptado	
T05	Salida en_curso sin chequeo aprobado	se rechaza	rechazado	La salida no tiene un chequeo aprobado
T06	Ítem desaprobado: chequeo rechazado y salida bloqueada	se acepta	aceptado	
T07	Salida con VTV vencida	se rechaza	rechazado	El vehículo AA333CC no tiene vtv vigente
T08	Chofer con licencia vencida	se rechaza	rechazado	La licencia del chofer CH002 está vencida
T09	Mismo vehículo en dos salidas a la vez	se rechaza	rechazado	conflicting key value violates exclusion constraint "salida_vehiculo_sin_solape"
T10	Kilometraje negativo	se rechaza	rechazado	new row for relation "salida" violates check constraint "salida_kilometraje_check"
T11	Mantenimiento abierto con repuesto: vehículo en_mantenimiento y stock descontado	se acepta	aceptado	
T11b	Cierre del mantenimiento: el vehículo vuelve a operativo	se acepta	aceptado	
T12	Repuesto mayor al stock disponible	se rechaza	rechazado	new row for relation "repuesto" violates check constraint "repuesto_stock_check"
T13	Transición de estado inválida (en_curso a solicitada)	se rechaza	rechazado	Transición de estado no permitida: en_curso -> solicitada
T14	Vista de vencimientos próximos	se acepta	aceptado	
T15	Documento duplicado para el mismo vehículo y tipo	se rechaza	rechazado	duplicate key value violates unique constraint "documentacion_patente_tipo_key"
T16	Cerrar salida con regreso anterior a la salida	se rechaza	rechazado	new row for relation "salida" violates check constraint "salida_check"
RV01	Insertar una salida ya finalizada, sin chequeo	se rechaza	rechazado	Una salida nueva debe crearse en estado solicitada (recibido: finalizada)
RV02	Insertar una salida directamente en_curso	se rechaza	rechazado	Una salida nueva debe crearse en estado solicitada (recibido: en_curso)
RV03	Chequeo rechazado deja el vehículo fuera de servicio	se acepta	aceptado	

Pruebas de las funciones nuevas

23 de 23 pruebas dieron el resultado esperado, incluyendo los datos rechazados.

Prueba	Descripción	Esperado	Obtenido	Mensaje del motor
S01	Alta de plan preventivo	se acepta	aceptado	
S02	Plan con intervalo de kilometraje en cero	se rechaza	rechazado	new row for relation "plan_preventivo" violates check constraint "plan_preventivo_cada_km_check"
S03	Plan duplicado para el mismo vehículo y descripción	se rechaza	rechazado	duplicate key value violates unique constraint "plan_preventivo_patente_descripcion_key"
S04	Flujo completo de salida y cierre suma los kilómetros al vehículo	se acepta	aceptado	
S05	Vista de preventivos pendientes: solo el vehículo que superó el intervalo	se acepta	aceptado	
S06	Abrir mantenimiento preventivo desde el plan	se acepta	aceptado	
S07	Usar repuesto genera movimiento de consumo y descuenta stock	se acepta	aceptado	
S08	Usar más repuesto del que hay en stock	se rechaza	rechazado	new row for relation "repuesto" violates check constraint "repuesto_stock_check"
S09	Ingreso de repuesto al pañol suma stock	se acepta	aceptado	
S10	Movimiento de ingreso con cantidad negativa	se rechaza	rechazado	new row for relation "movimiento_stock" violates check constraint "movimiento_stock_check"
S11	Movimiento de consumo con cantidad positiva	se rechaza	rechazado	new row for relation "movimiento_stock" violates check constraint "movimiento_stock_check1"
S12	Movimiento sobre un repuesto inexistente	se rechaza	rechazado	insert or update on table "movimiento_stock" violates foreign key constraint "movimiento_stock_codigo_fkey"
S13	Repuesto bajo el punto de reposición aparece en la vista	se acepta	aceptado	
S14	Generación de alertas de vencimiento	se acepta	aceptado	
S15	Generar alertas dos veces no duplica	se acepta	aceptado	
S16	Alerta sin documento ni chofer	se rechaza	rechazado	new row for relation "alerta_vencimiento" violates check constraint "alerta_vencimiento_check"
S17	Alerta con documento y chofer a la vez	se rechaza	rechazado	new row for relation "alerta_vencimiento" violates check constraint "alerta_vencimiento_check"
S18	Atender una alerta pendiente	se acepta	aceptado	
S19	Atender una alerta que ya fue atendida	se rechaza	rechazado	La alerta no existe o ya fue atendida
S20	Iniciar una salida cuyo chequeo fue rechazado	se rechaza	rechazado	Transición de estado no permitida: bloqueada -> en_curso
S21	Cerrar el mantenimiento devuelve el vehículo a operativo	se acepta	aceptado	
S22	Cerrar un mantenimiento ya cerrado	se rechaza	rechazado	El mantenimiento no existe o ya está cerrado
S23	Indicadores de flota	se acepta	aceptado	

Código de las pruebas

```
-- Pruebas etapa 2, Caso 1. Se ejecutan después de la batería completa de la etapa 1 (regresión).

-- @test S01 | Alta de plan preventivo | expect=ok
insert into plan_preventivo (patente, descripcion, cada_km) values ('AA222BB', 'Service de 5.000 km', 5000);
-- @end

-- @test S02 | Plan con intervalo de kilometraje en cero | expect=reject
insert into plan_preventivo (patente, descripcion, cada_km) values ('AA222BB', 'Plan inválido', 0);
-- @end

-- @test S03 | Plan duplicado para el mismo vehículo y descripción | expect=reject
insert into plan_preventivo (patente, descripcion, cada_km) values ('AA111AA', 'Service de 10.000 km', 8000);
-- @end

-- @test S04 | Flujo completo de salida y cierre suma los kilómetros al vehículo | expect=ok
do $$
declare s uuid; r text;
begin
  s := solicitar_salida('AA444DD', 'CH004', 'Base', 'Campamento');
  r := registrar_chequeo(s, '[{"descripcion":"Frenos","aprobado":true},{"descripcion":"Luces","aprobado":true}]');
  if r <> 'aprobado' then raise exception 'El chequeo debía quedar aprobado y quedó %', r; end if;
  perform iniciar_salida(s);
  perform cerrar_salida(s, 300);
end $$;
-- @end

-- @test S05 | Vista de preventivos pendientes: solo el vehículo que superó el intervalo | expect=ok
select 1;
-- @end

-- @test S06 | Abrir mantenimiento preventivo desde el plan | expect=ok
select abrir_mantenimiento_preventivo((select id_plan from plan_preventivo where patente = 'AA333CC'), 'ME001');
-- @end

-- @test S07 | Usar repuesto genera movimiento de consumo y descuenta stock | expect=ok
select usar_repuesto((select id_mant from mantenimiento where patente = 'AA333CC'), 'REP-PAST-02', 3);
-- @end

-- @test S08 | Usar más repuesto del que hay en stock | expect=reject
select usar_repuesto((select id_mant from mantenimiento where patente = 'AA333CC'), 'REP-PAST-02', 5);
-- @end

-- @test S09 | Ingreso de repuesto al pañol suma stock | expect=ok
select ingresar_repuesto('REP-PAST-02', 10);
-- @end

-- @test S10 | Movimiento de ingreso con cantidad negativa | expect=reject
insert into movimiento_stock (codigo, tipo, cantidad) values ('REP-PAST-02', 'ingreso', -2);
-- @end

-- @test S11 | Movimiento de consumo con cantidad positiva | expect=reject
insert into movimiento_stock (codigo, tipo, cantidad) values ('REP-PAST-02', 'consumo', 2);
-- @end

-- @test S12 | Movimiento sobre un repuesto inexistente | expect=reject
insert into movimiento_stock (codigo, tipo, cantidad) values ('NO-EXISTE', 'ingreso', 5);
-- @end

-- @test S13 | Repuesto bajo el punto de reposición aparece en la vista | expect=ok
select usar_repuesto((select id_mant from mantenimiento where patente = 'AA333CC'), 'REP-FILT-01', 5);
-- @end

-- @test S14 | Generación de alertas de vencimiento | expect=ok
do $$ begin
  if generar_alertas(30) <> 4 then raise exception 'Se esperaban 4 alertas nuevas'; end if;
end $$;
-- @end

-- @test S15 | Generar alertas dos veces no duplica | expect=ok
do $$ begin
  if generar_alertas(30) <> 0 then raise exception 'La segunda generación no debía crear alertas'; end if;
end $$;
-- @end

-- @test S16 | Alerta sin documento ni chofer | expect=reject
insert into alerta_vencimiento (vencimiento) values (current_date);
-- @end

-- @test S17 | Alerta con documento y chofer a la vez | expect=reject
insert into alerta_vencimiento (id_doc, legajo_chofer, vencimiento) values ((select id_doc from documentacion limit
    1), 'CH001', current_date + 3);
-- @end
```

```
-- @test S18 | Atender una alerta pendiente | expect=ok
select atender_alerta((select id_alerta from alerta_vencimiento where legajo_chofer = 'CH002'), 'Supervisor
    ficticio');
-- @end

-- @test S19 | Atender una alerta que ya fue atendida | expect=reject
select atender_alerta((select id_alerta from alerta_vencimiento where legajo_chofer = 'CH002'), 'Otro supervisor');
-- @end

-- @test S20 | Iniciar una salida cuyo chequeo fue rechazado | expect=reject
do $$
declare s uuid;
begin
  s := solicitar_salida('AA666FF', 'CH004', 'Base', 'Planta');
  perform registrar_chequeo(s, '[{"descripcion":"Frenos","aprobado":false}]');
  perform iniciar_salida(s);
end $$;
-- @end

-- @test S21 | Cerrar el mantenimiento devuelve el vehículo a operativo | expect=ok
select cerrar_mantenimiento((select id_mant from mantenimiento where patente = 'AA333CC'));
-- @end

-- @test S22 | Cerrar un mantenimiento ya cerrado | expect=reject
select cerrar_mantenimiento((select id_mant from mantenimiento where patente = 'AA333CC'));
-- @end

-- @test S23 | Indicadores de flota | expect=ok
select 1;
-- @end
```

5. Aplicación web

5.1 Prompt

Se usó el documento descriptivo completo (sección 2) precedido por este pedido:

```
Construí la aplicación web descripta en el documento anterior con React, TypeScript y Vite, en español. Usá
    supabase-js con las variables VITE_SUPABASE_URL y VITE_SUPABASE_ANON_KEY. Un selector de rol arriba a la derecha
     define qué pantallas se ven. Cada acción debe llamar a las funciones (RPC) de la base y mostrar el resultado o
    el error del motor en un aviso. No dupliques reglas del negocio en la aplicación: la base las hace cumplir.
    Entregá también el README, .env.example sin valores y .gitignore que excluya .env.
```

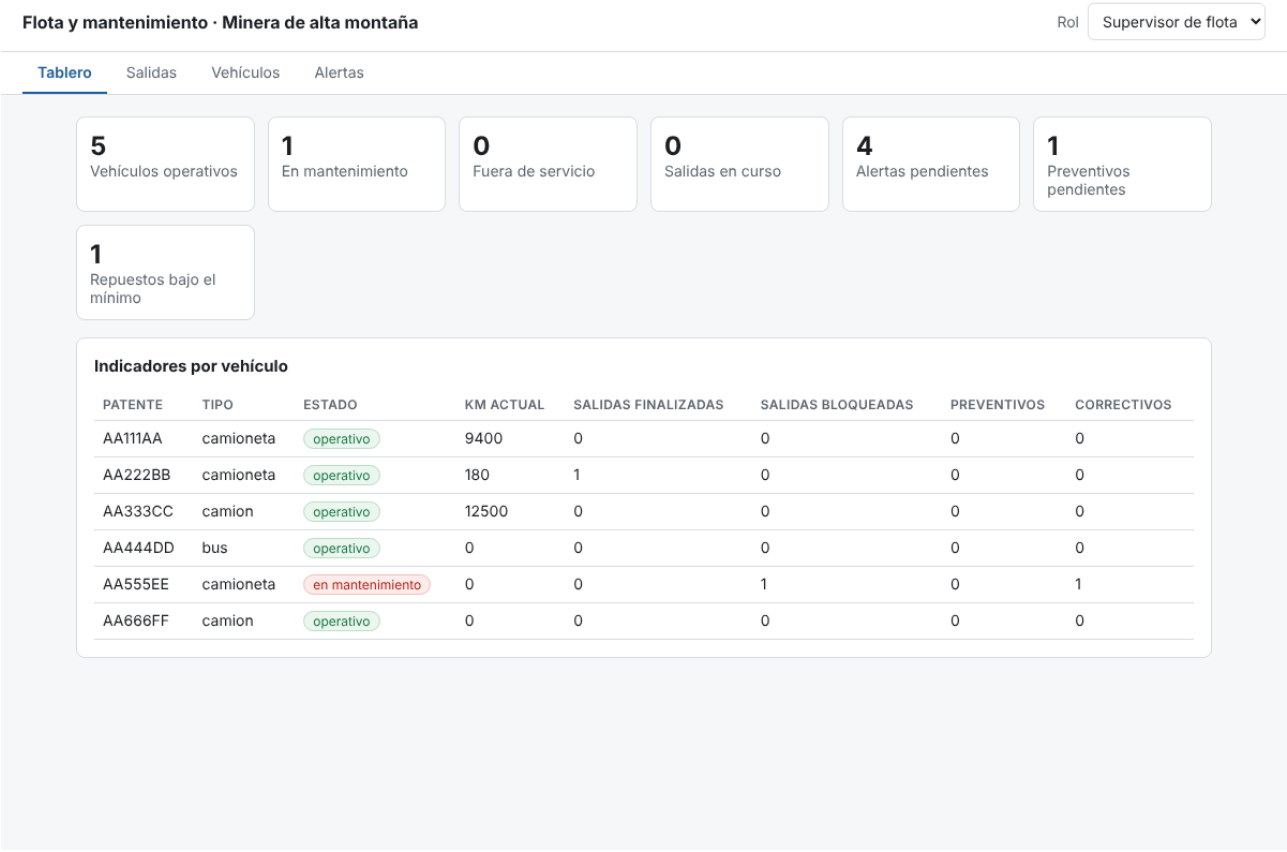
5.2 Correcciones posteriores

La primera versión generada se probó contra la base real y se corrigió lo siguiente.

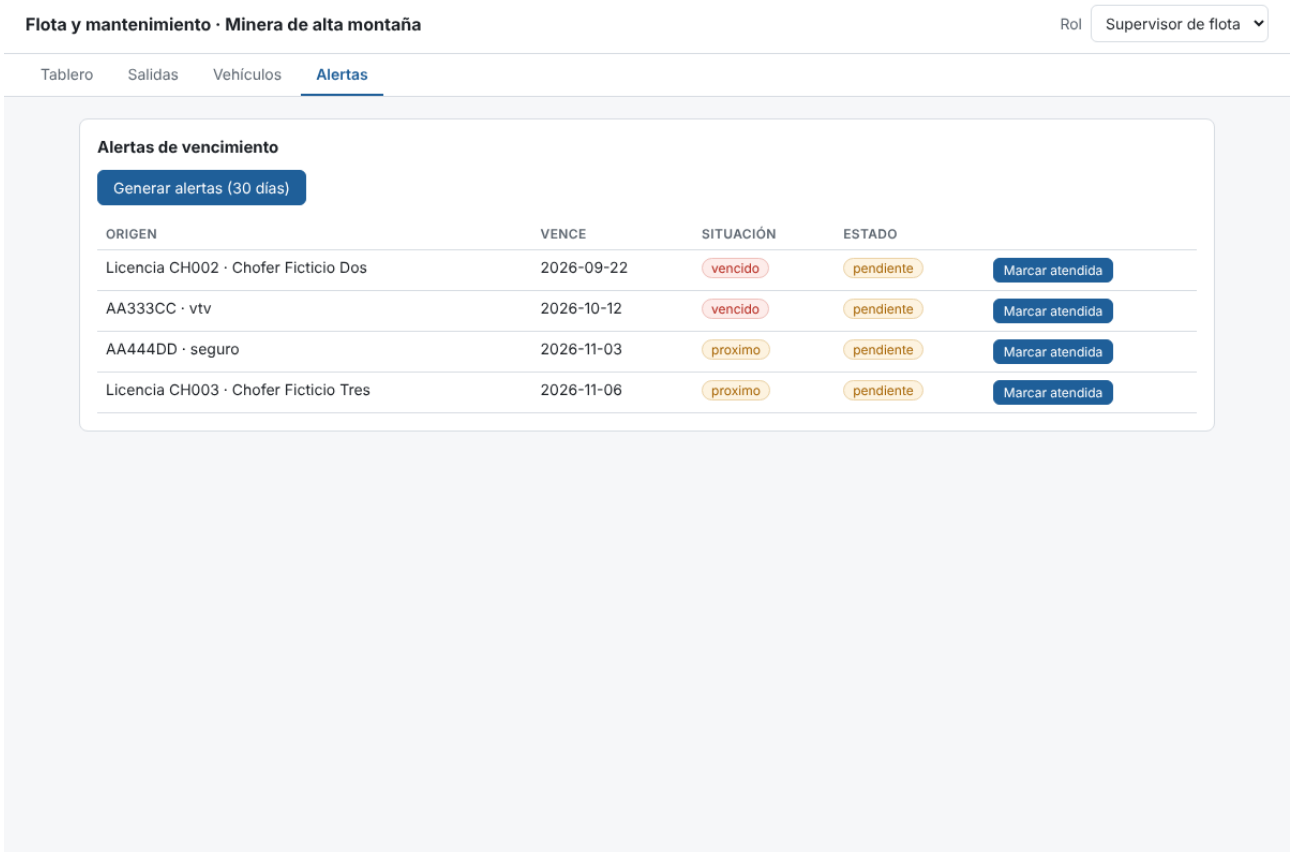
Problema	Corrección
Aviso de resultado que desaparecía antes de tiempo	Al hacer dos acciones seguidas, el temporizador del primer aviso borraba el segundo. Se guardó el temporizador y se cancela el anterior al mostrar un aviso nuevo.
Datos de pantalla vacíos con la configuración de seguridad de Supabase	Las llamadas con la clave anónima exigen permisos sobre tablas y funciones. Se definieron las políticas RLS y los permisos en la migración, y se verificó la aplicación con una clave firmada.
Relaciones anidadas mal tipadas	La compilación con TypeScript rechazaba expresiones siempre verdaderas en dos columnas de tablas. Se reemplazaron por el componente correcto.
Error de la base mostrado en pantalla	Las reglas del negocio rechazan operaciones (por ejemplo salir con VTV vencida). La aplicación muestra el mensaje del motor en un aviso rojo en lugar de fallar en silencio.

5.3 Capturas

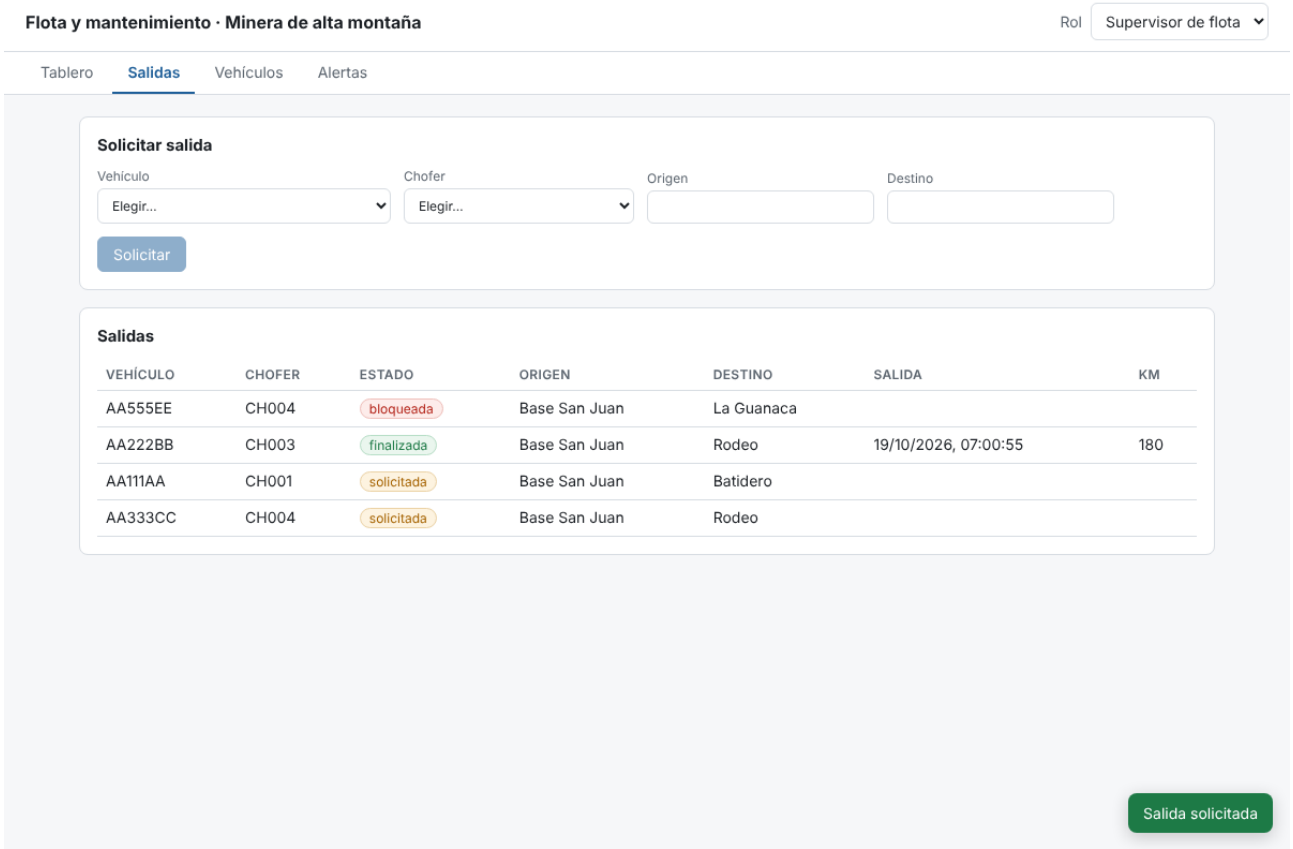
Capturas de la aplicación funcionando contra la base de la etapa 2, con datos ficticios. Incluyen un rechazo de la base mostrado en pantalla.



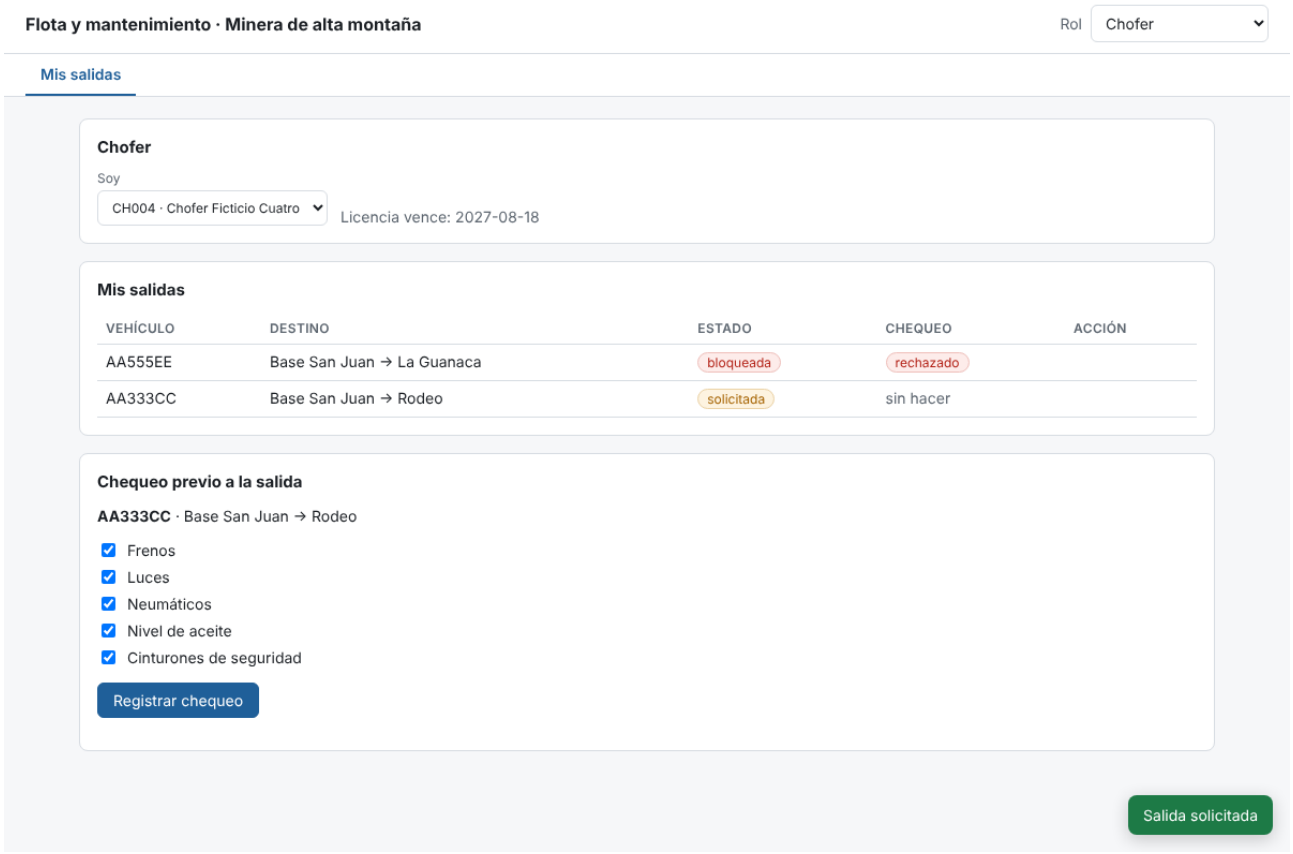
supervisor tablero



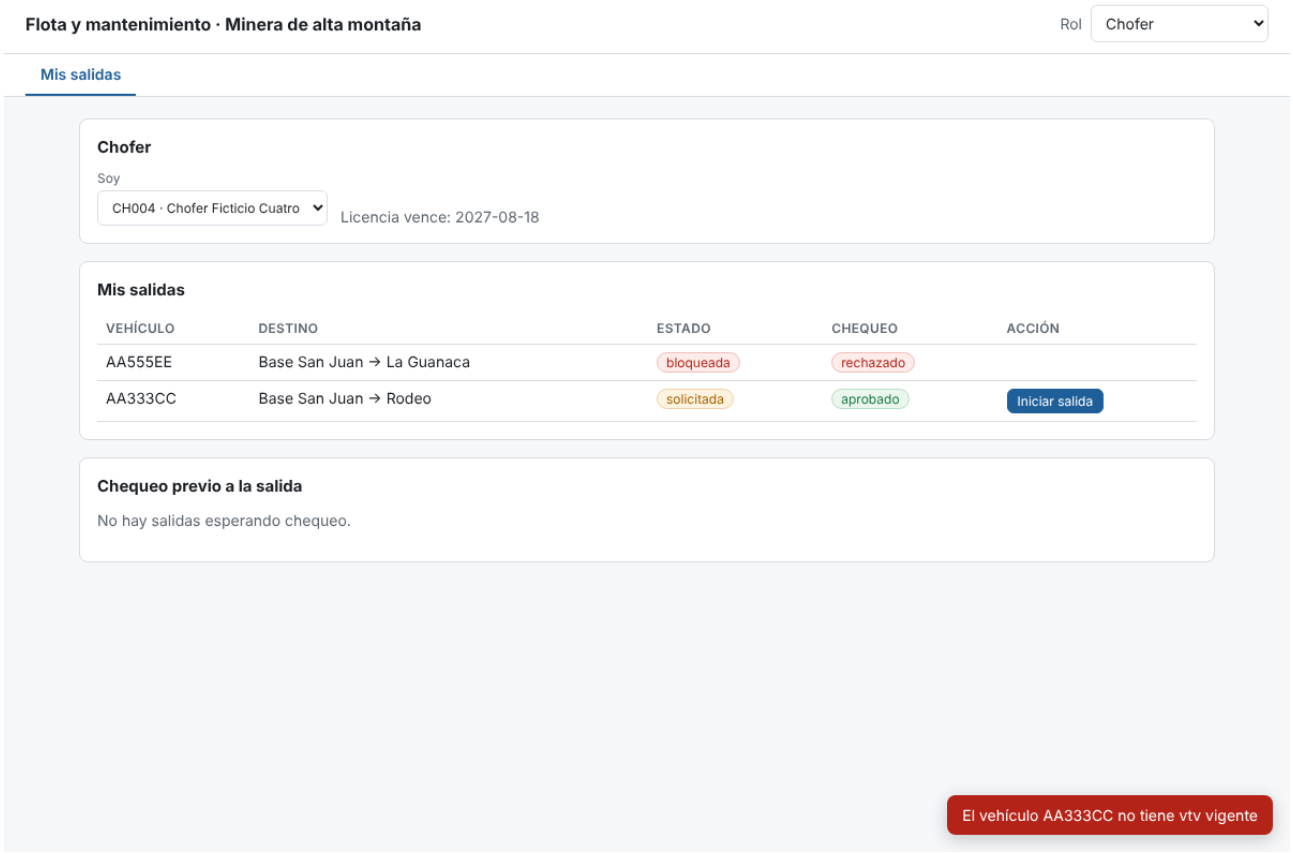
supervisor alertas



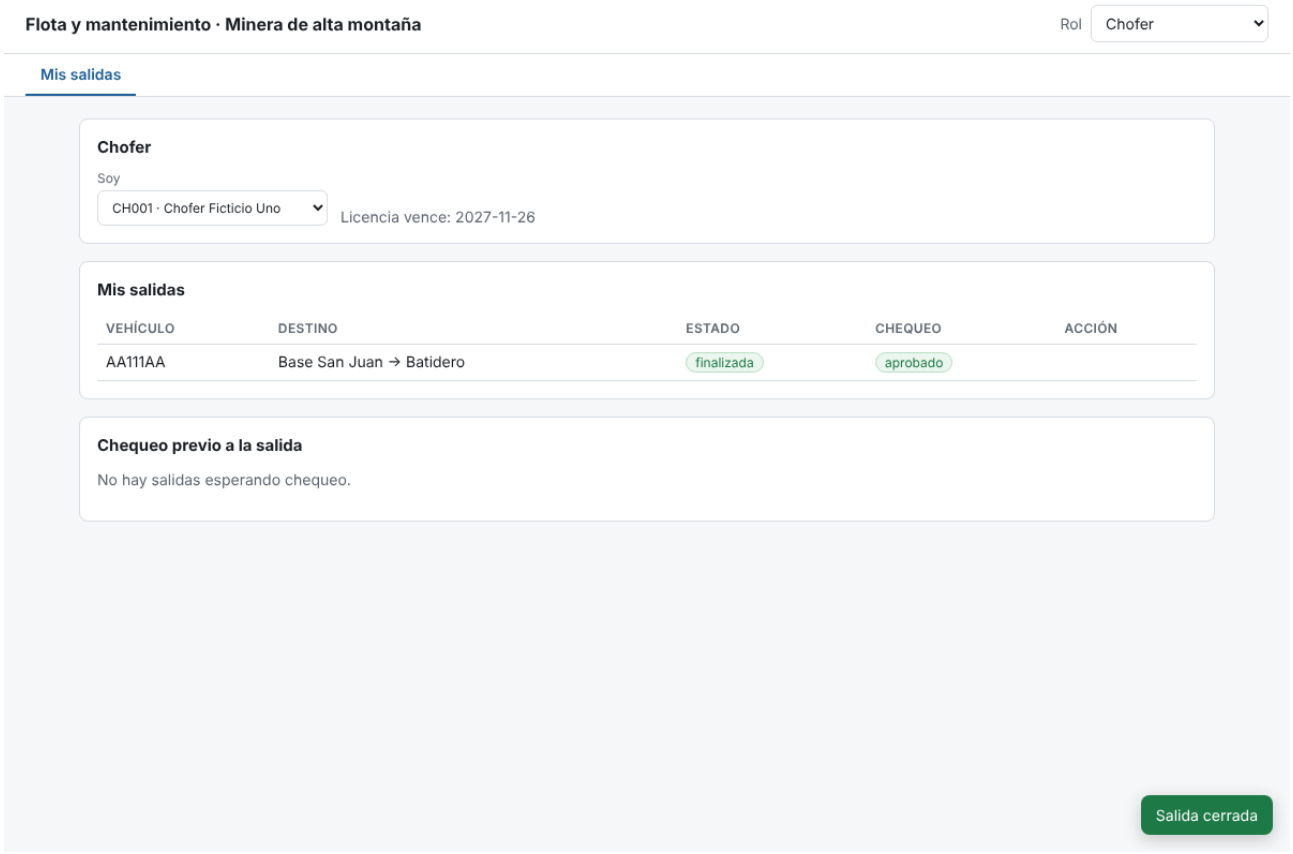
supervisor salidas



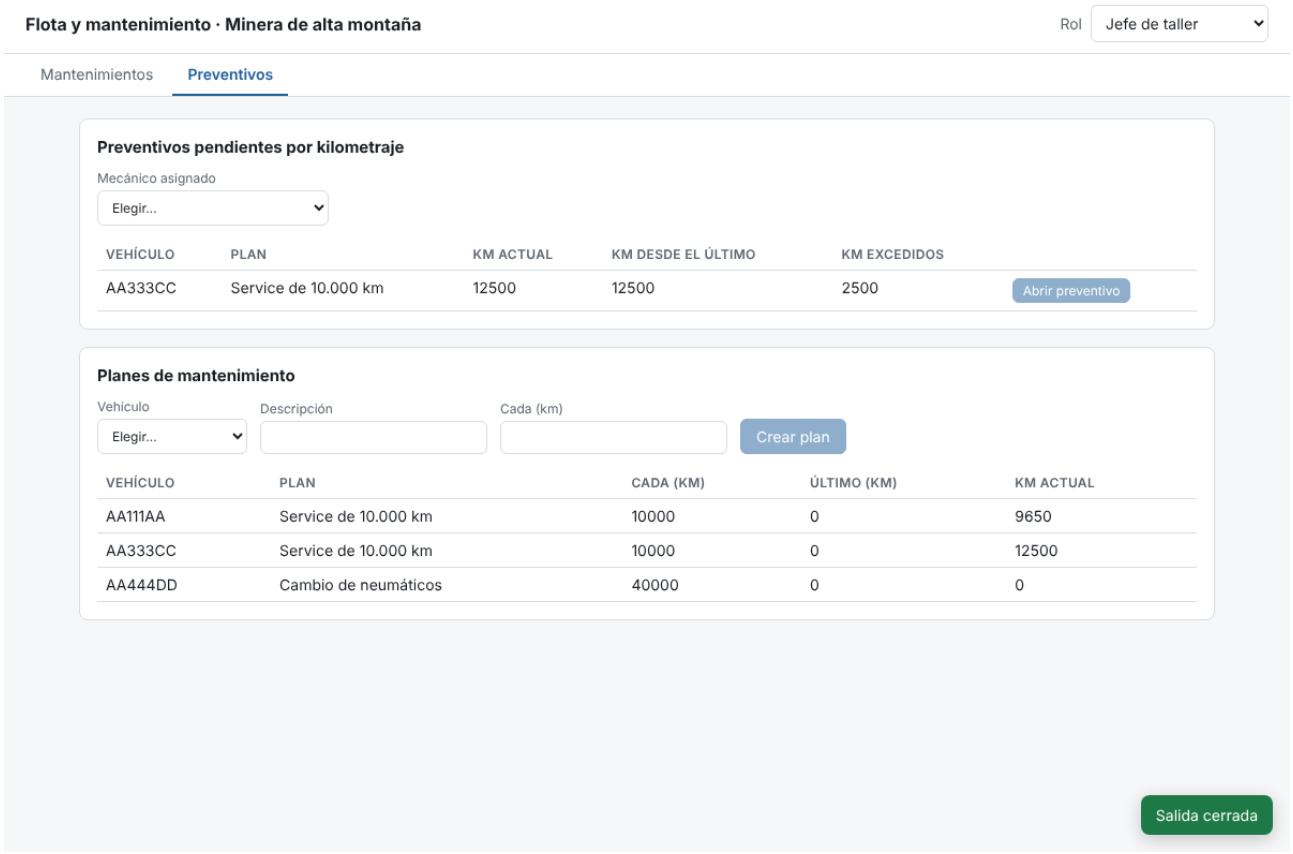
chofer chequeo



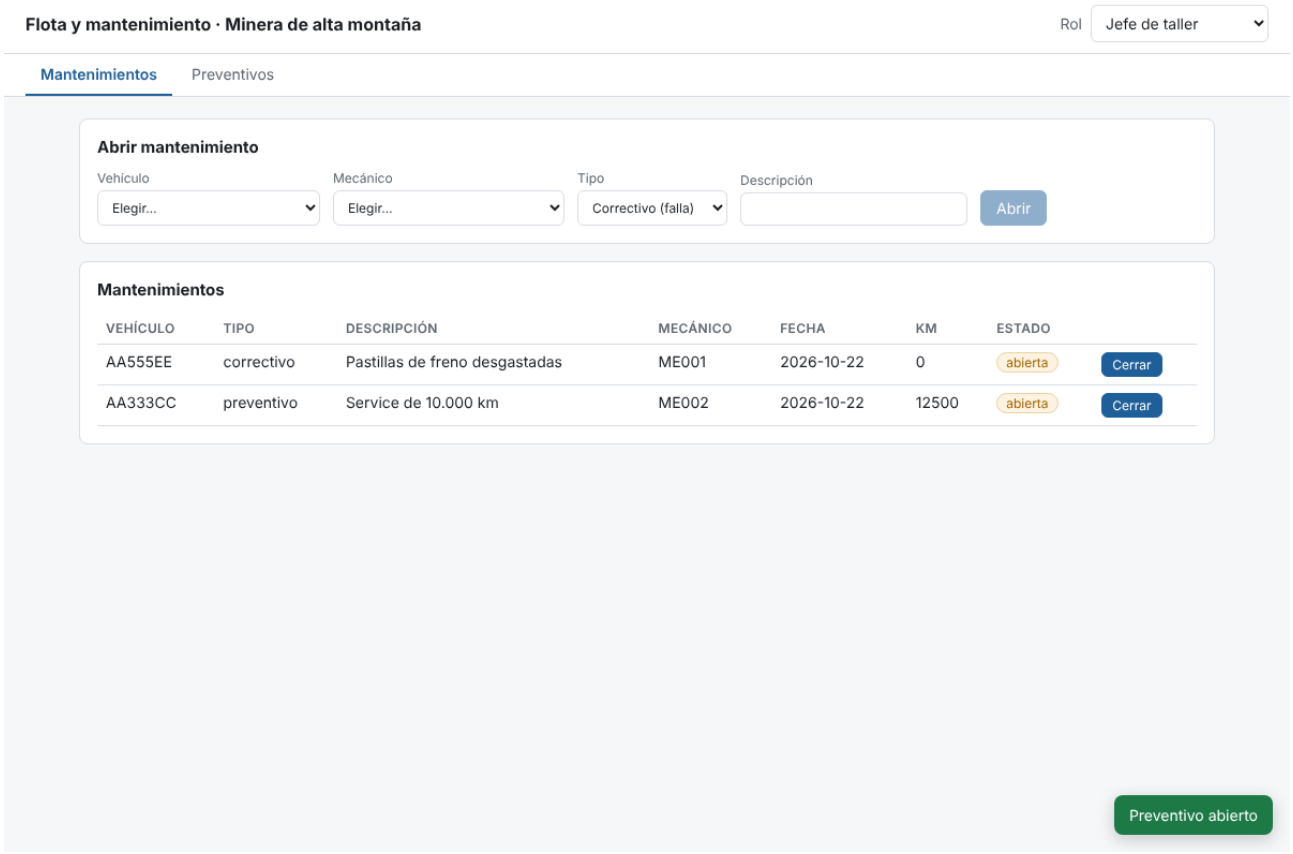
chofer error vtv



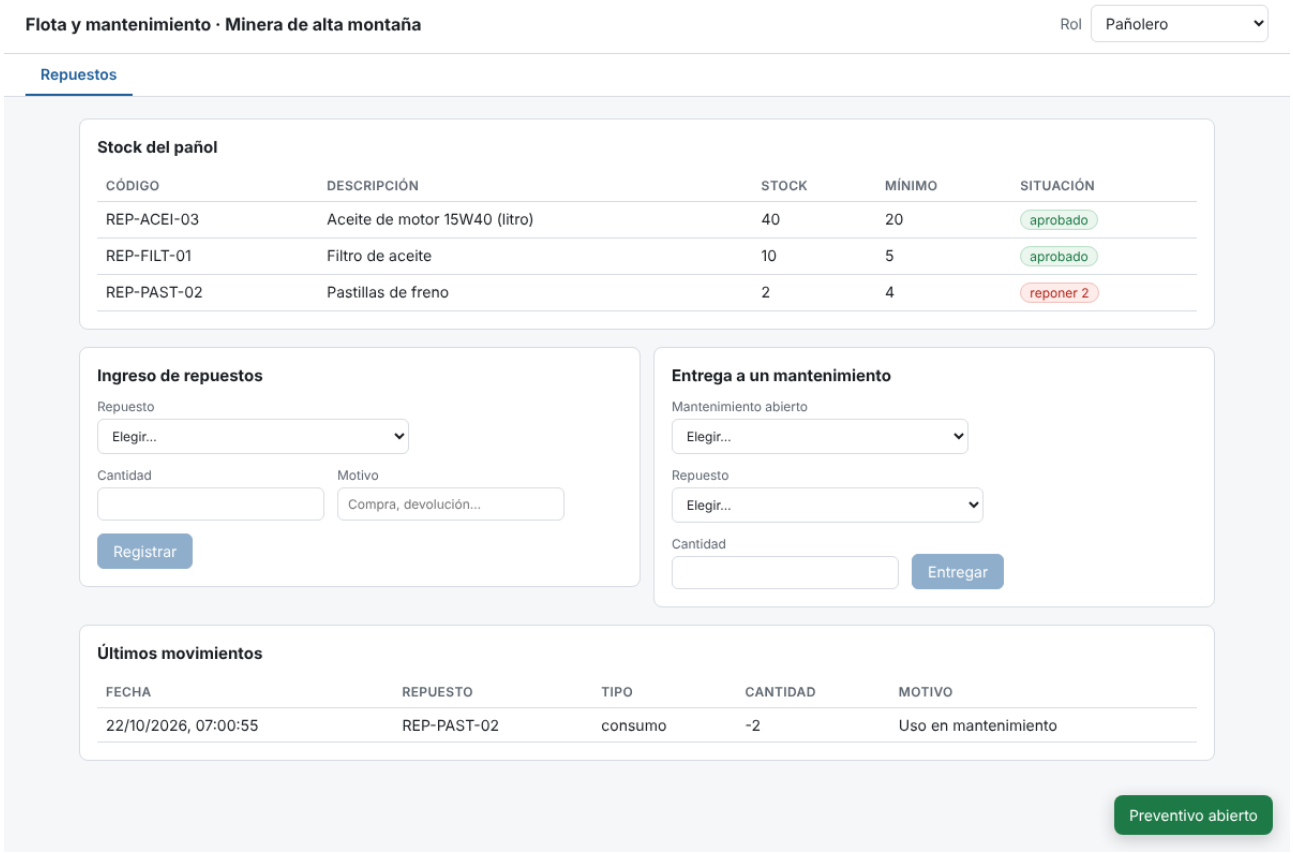
chofer salida cerrada



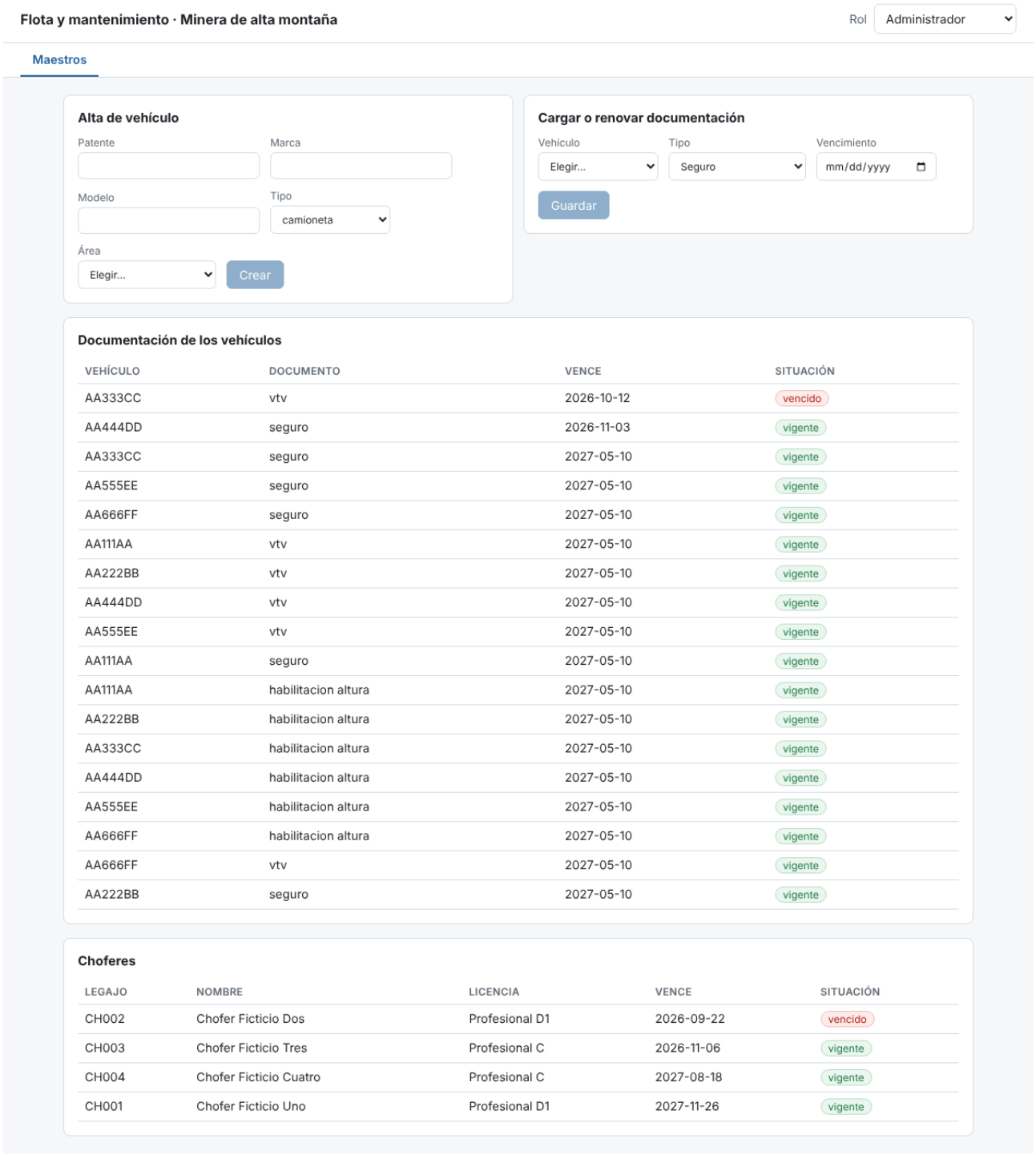
taller preventivos



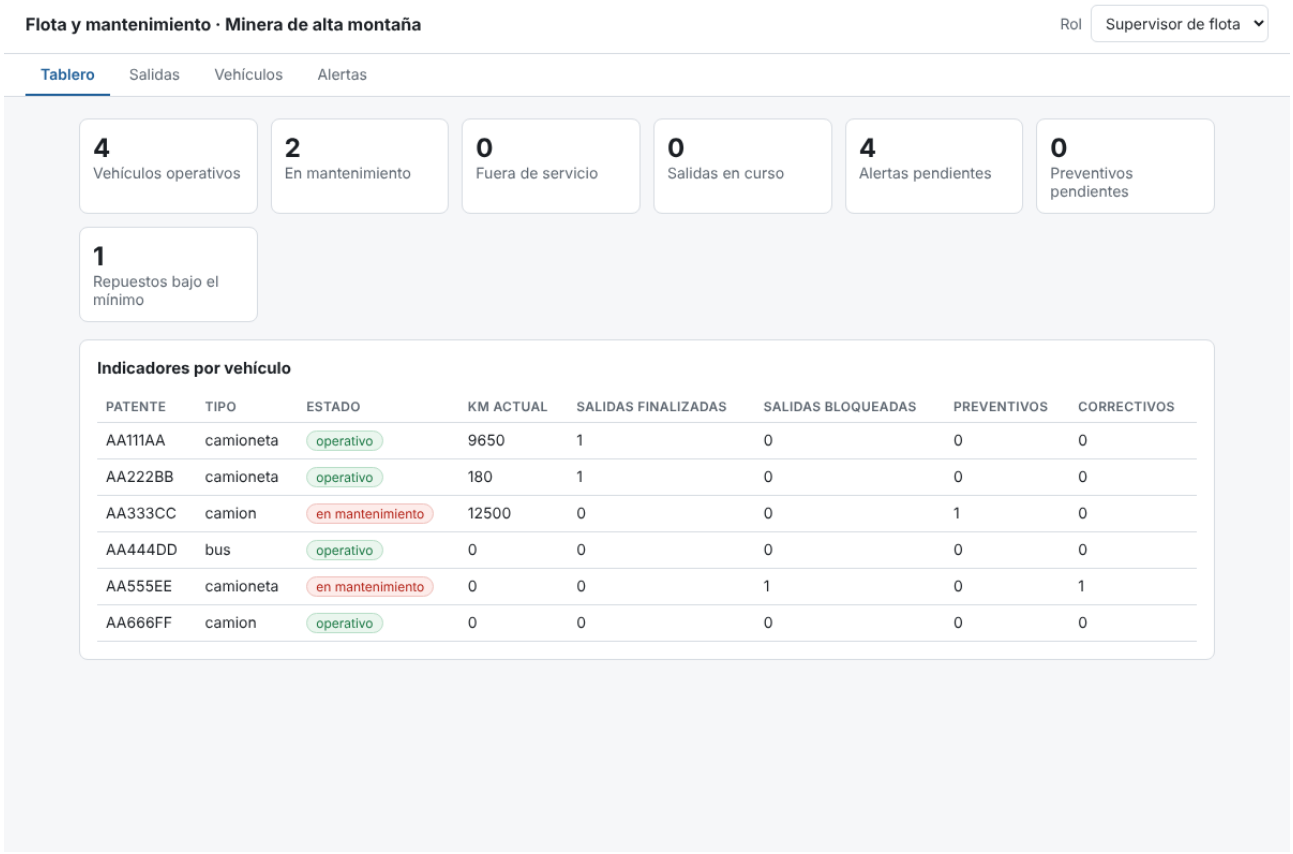
taller mantenimientos



panol repuestos



admin maestros



supervisor tablero final

6. Repositorio

Nombre sugerido: `tp-caso1-flota`. Enlace: _completar después de subirlo a GitHub_. Contenido: código de la aplicación (`src`), `docs/` con documento descriptivo, MER, modelo relacional y scripts SQL, `supabase/migrations`, `README.md`, `.env.example` (solo los nombres de las variables, sin valores) y `.gitignore` que excluye `.env`. Las credenciales de Supabase no se suben; se cargan en un `.env` local y en las variables de entorno de Vercel.